

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Globalization;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using DataLink;

namespace LINK
{
    public partial class Form1 : Form
    {
        #region Campos privados

        private TQS.Drawing2D.DrawingComponent dc; // Componente de desenho
        TQS.

        private Data data; // Dados do modelo.

        private readonly List<Button> navigationButtons = new List<Button>(); //
        Botões de navegação.

        private StatusStrip statusStrip; // Barra de status do rodapé.

        private ToolStripStatusLabel statusLabel; // Rótulo de status do rodapé.

        private RichTextBox reportBox; // Exibe o memorial.

        private Panel schematicPanel; // Painei do esquema 2D.

        private Button calculateButton; // Aciona o cálculo.

        private Button generateReportButton; // Gera o relatório.

        private TableLayoutPanel mainLayout; // Grade principal (navegação, centro,
        lateral).

```

```
private Panel rightPanelHost; // Paine lateral para desenho ou resultados.

private Panel view3DPanel; // Placeholder da visualização 3D.

private Panel overlayPanel; // Paine que cobre a tela principal.

private TableLayoutPanel overlayHeader; // Cabeçalho do overlay.

private TableLayoutPanel overlayContent; // Corpo do overlay.

private Button overlayCloseButton; // Botão fechar overlay.

private Label overlayTitle; // Título do overlay.

private Panel overlayPlaceholder; // Placeholder para telas não-geom.

private string overlayMode; // "Geometry" ou outros.

private Button overlayMaxButton; // Botão maximizar/restaurar overlay.

private bool overlayMaximized; // Estado de tamanho do overlay.

private Rectangle overlayRestoreBounds; // Tamanho/posição antes de
maximizar.

private bool overlayDragging; // Flag de arrasto do overlay.

private Point overlayDragOffset; // Offset para arrasto.

private Font captionIconFont; // Fonte de ícones do cabeçalho.

private bool linkTypeSelected; // Indica se já escolheu o tipo de ligação.

private GeometryParameters geometryParams = new GeometryParameters();
// Parâmetros detalhados de geometria.

private bool geometryInputsUpdating; // Evita laços durante sincronização de
campos.

private ComboBox beamProfileCombo;

private ComboBox columnProfileCombo;

private TextBox beamHeightBox;

private TextBox beamFlangeBox;

private TextBox beamWebBox;

private TextBox beamFlangeWidthBox;

private TextBox columnHeightBox;

private TextBox columnFlangeBox;
```

```

private TextBox columnWebBox;

private TextBox columnFlangeWidthBox;

private TextBox plateThicknessBox;

private TextBox plateWidthBox;

private TextBox plateEdgeHorizontalBox;

private TextBox plateEdgeTopBox;

private TextBox plateEdgeBottomBox;

private TextBox boltCountBox;

private TextBox boltSpacingBox;

private TextBox boltEdgeTopBox;

private TextBox boltGaugeBox;

private TextBox boltDiameterBox;

private TextBox weldSizeBox;

private Panel sideViewPanel;

private Panel frontViewPanel;

private Helix3DHost view3DHost;

private HelixSceneBuilder helixSceneBuilder;

private Control geometryInputsLayout;

private Panel parametersHost;

private static readonly IReadOnlyList<ProfileDefinition> ProfileLibrary =
ProfileLibraryData.Profiles;

#endregion

public Form1()
{
    InitializeComponent();

    EngineeringStyle.ApplyModernStyle(this);

    AutoScaleMode = AutoScaleMode.Dpi;

```

```

AutoScaleDimensions = new SizeF(96F, 96F);

data = new Data();

helixSceneBuilder = new HelixSceneBuilder();

ConfigureUI();

ConfigureLayout();

LoadGeometryToInputs();
}

private void ConfigureUI()
{
    if (menuStrip1 != null)
    {
        menuStrip1.Dock = DockStyle.Top;

        MainMenuStrip = menuStrip1;
    }

    if (toolStrip1 != null)
    {
        toolStrip1.Items.Clear();

        toolStrip1.ImageScalingSize = new Size(20, 20);

        toolStrip1.GripStyle = ToolStripGripStyle.Hidden;

        toolStrip1.RenderMode = ToolStripRenderMode.System;

        toolStrip1.Dock = DockStyle.Top;

        toolStrip1.Items.Add(CreateIconButton("Geometria",
LoadIcon("IconGeometria", SystemIcons.Information), OnToolbarStageClicked,
"Geometria"));

```

```
toolStrip1.Items.Add(CreateIconButton("Propriedades",  
LoadIcon("IconPropriedades", SystemIcons.Question), OnToolbarStageClicked,  
"Propriedades"));
```

```
toolStrip1.Items.Add(CreateIconButton("Cargas", LoadIcon("IconCargas",  
SystemIcons.Warning), OnToolbarStageClicked, "Cargas"));
```

```
toolStrip1.Items.Add(CreateIconButton("Critérios",  
LoadIcon("IconCritérios", SystemIcons.Shield), OnToolbarStageClicked,  
"Critérios"));
```

```
toolStrip1.Items.Add(CreateIconButton("Resultados",  
LoadIcon("IconResultados", SystemIcons.Application), OnToolbarStageClicked,  
"Resultados"));
```

```
toolStrip1.Items.Add(new ToolStripSeparator());
```

```
toolStrip1.Items.Add(CreateIconButton("VD", LoadIcon("IconVD",  
SystemIcons.Warning), OnToolbarStageClicked, "Esforço cortante VD"));
```

```
toolStrip1.Items.Add(CreateIconButton("ND", LoadIcon("IconND",  
SystemIcons.Shield), OnToolbarStageClicked, "Esforço normal ND"));
```

```
toolStrip1.Items.Add(CreateIconButton("MD", LoadIcon("IconMD",  
SystemIcons.Information), OnToolbarStageClicked, "Momento MD"));
```

```
toolStrip1.Items.Add(new ToolStripSeparator());
```

```
toolStrip1.Items.Add(CreateIconButton("Calcular", LoadIcon("IconRun",  
SystemIcons.Exclamation), OnToolbarStageClicked, "Calcular"));
```

```
toolStrip1.Items.Add(new ToolStripSeparator());
```

```
toolStrip1.Items.Add(CreateIconButton("ZoomIn", LoadIcon("IconZoomIn",  
SystemIcons.Asterisk), OnToolbarStageClicked, "Zoom in"));
```

```
toolStrip1.Items.Add(CreateIconButton("ZoomOut",  
LoadIcon("IconZoomOut", SystemIcons.Asterisk), OnToolbarStageClicked, "Zoom  
out"));
```

```
        toolStrip1.Items.Add(CreateIconButton("ZoomFit",  
LoadIcon("IconZoomFit", SystemIcons.Application), OnToolBarStageClicked,  
"Zoom extents"));
```

```
        toolStrip1.Items.Add(new ToolStripSeparator());
```

```
        toolStrip1.Items.Add(CreateIconButton("Login", LoadIcon("IconLogin",  
SystemIcons.WinLogo), OnToolBarStageClicked, "Credenciais / TQS"));
```

```
    }
```

```
}
```

```
private Image LoadIcon(string resourceKey, Icon fallbackIcon)
```

```
{
```

```
    try
```

```
    {
```

```
        var obj = Properties.Resources.ResourceManager.GetObject(resourceKey);
```

```
        if (obj is Bitmap bmp) return bmp;
```

```
        if (obj is Icon ico) return ico.ToBitmap();
```

```
    }
```

```
    catch
```

```
    {
```

```
    }
```

```
    return fallbackIcon?.ToBitmap();
```

```
}
```

```
private ToolStripButton CreateIconButton(string tag, Image icon, EventHandler  
onClick, string toolTip)
```

```
{
```

```
    var btn = new ToolStripButton
```

```
    {
```

```

        Tag = tag,
        Image = icon,
        DisplayStyle = ToolStripItemDisplayStyle.Image,
        ToolTipText = toolTip,
        AutoSize = true,
        Margin = new Padding(2)
    };
    btn.Click += onClick;
    return btn;
}

```

```

private void OnQuickActionClicked(object sender, EventArgs e)
{
    if (sender is ToolStripButton button)
    {
        ShowStatus($"Ação rápida: {button.Text} (placeholder)", false);
    }
}

```

```

private void OnToolbarStageClicked(object sender, EventArgs e)
{
    if (sender is ToolStripButton button)
    {
        var tag = button.Tag as string;

        if ((tag == "Geometria" || tag == "Propriedades" || tag == "Cargas" || tag ==
"Critérios" || tag == "Resultados") && !linkTypeSelected)
        {

```

```
        ShowStatus("Escolha primeiro o tipo de ligação na coluna à esquerda.",
true);

        return;
    }

    switch (tag)
    {
        case "Geometria":

            tabControl1.SelectedTab = tabPage1;

            ShowGeometryOverlay();

            break;

        case "Propriedades":

            ShowGenericOverlay("Propriedades", "Defina materiais, tabelas e
parâmetros gerais aqui.");

            break;

        case "Cargas":

            ShowGenericOverlay("Cargas", "Cadastro de casos e combinações de
carga.");

            break;

        case "Critérios":

            ShowGenericOverlay("Critérios", "Configuração de critérios e
verificações.");

            break;

        case "Resultados":

            ShowGenericOverlay("Resultados", "Resultados e relatórios gerados.");

            break;

        case "VD":

            ShowGenericOverlay("VD", "Esforço cortante de cálculo (VD).");

            break;
```



```

        case "ND":
            ShowGenericOverlay("ND", "Esforço normal de cálculo (ND).");
            break;
        case "MD":
            ShowGenericOverlay("MD", "Momento de cálculo (MD).");
            break;
        case "Calcular":
            OnCalculateClicked(sender, EventArgs.Empty);
            break;
        case "ZoomIn":
        case "ZoomOut":
        case "ZoomFit":
            ShowStatus($"Ação de zoom: {tag} (placeholder)", false);
            break;
        case "Login":
            ShowStatus("Ação de login/acesso (placeholder)", false);
            break;
    }
}
}

```

```

private void button1_Click(object sender, EventArgs e)
{
    // código TQS comentado
}

```

```

private void button2_Click(object sender, EventArgs e)
{

```

```

Process cmd = new Process();

cmd.StartInfo.FileName = "NULF.EXE";

cmd.StartInfo.Arguments = "";

cmd.StartInfo.CreateNoWindow = true;

cmd.StartInfo.UseShellExecute = true;

cmd.StartInfo.Verb = "runas";

cmd.StartInfo.WindowStyle = ProcessWindowStyle.Normal;

cmd.Start();

cmd.WaitForExit();


InitializeComponent2();
}

```

```

private void InitializeComponent2()
{
    if (TQS.TQSTST.delta() != 0)
    {
        // lógica TQS comentada
    }
    else
    {
        // lógica TQS comentada
    }
}

```

```

private void ConfigureLayout()
{
    if (tableLayoutPanel1 != null)

```

```
{  
    tableLayoutPanel1.RowCount = 4;  
    tableLayoutPanel1.RowStyles.Clear();  
    tableLayoutPanel1.RowStyles.Add(new RowStyle(SizeType.AutoSize));  
    tableLayoutPanel1.RowStyles.Add(new RowStyle(SizeType.AutoSize));  
    tableLayoutPanel1.RowStyles.Add(new RowStyle(SizeType.Percent, 100F));  
    tableLayoutPanel1.RowStyles.Add(new RowStyle(SizeType.AutoSize));  
  
    if (menuStrip1 != null) tableLayoutPanel1.SetRow(menuStrip1, 0);  
    if (toolStrip1 != null) tableLayoutPanel1.SetRow(toolStrip1, 1);  
}
```

```
ConfigureNavigation();
```

```
ConfigureTabs();
```

```
BuildMainLayout();
```

```
ConfigureFooter();
```

```
CreateOverlay();
```

```
tabControl1.SelectedIndexChanged -= OnTabChanged;
```

```
tabControl1.SelectedIndexChanged += OnTabChanged;
```

```
if (navigationButtons.Any())
```

```
{  
    SetActiveNavigationButton(navigationButtons.First());  
}
```

```
EnsureInputsLayoutOnSelectedTab();
```

```
UpdateRightPanelContent();
```

```
}
```

```
private void BuildMainLayout()
```

```
{
```

```
    if (tableLayoutPanel1 == null)
```

```
    {
```

```
        return;
```

```
    }
```

```
    if (mainLayout != null && tableLayoutPanel1.Controls.Contains(mainLayout))
```

```
    {
```

```
        tableLayoutPanel1.Controls.Remove(mainLayout);
```

```
    }
```

```
    if (tableLayoutPanel1.Controls.Contains(splitContainer1))
```

```
    {
```

```
        tableLayoutPanel1.Controls.Remove(splitContainer1);
```

```
    }
```

```
    mainLayout = new TableLayoutPanel
```

```
    {
```

```
        Dock = DockStyle.Fill,
```

```
        ColumnCount = 3,
```

```
        RowCount = 1,
```

```
        Padding = new Padding(6),
```

```
        Margin = Padding.Empty
```

```
    };
```

```
    // Sidebar slightly wider to improve usability on modern monitors
```

```

mainLayout.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 210f));
// Give more space to center inputs and reasonable space to 2D/3D preview
mainLayout.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 60f));
mainLayout.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 40f));
mainLayout.RowStyles.Add(new RowStyle(SizeType.Percent, 100f));

// Sidebar host (tableLayoutPanel2) - ensure it can scroll when many items
tableLayoutPanel2.Dock = DockStyle.Fill;
tableLayoutPanel2.Margin = Padding.Empty;
tableLayoutPanel2.AutoScroll = true;
tableLayoutPanel2.MinimumSize = new Size(180, 0);
mainLayout.Controls.Add(tableLayoutPanel2, 0, 0);

if (parametersHost == null)
{
    parametersHost = new Panel { Dock = DockStyle.Fill, Padding = new
Padding(8), AutoScroll = true };
}

parametersHost.Controls.Clear();
var inputs = GetSharedInputsLayout();
inputs.Dock = DockStyle.Fill;
parametersHost.Controls.Add(inputs);
parametersHost.MinimumSize = new Size(250, 0);
mainLayout.Controls.Add(parametersHost, 1, 0);

if (rightPanelHost == null)
{
    rightPanelHost = new Panel { Dock = DockStyle.Fill, Padding = new
Padding(8), BackColor = Color.WhiteSmoke };
}

```

```

    }

    rightPanelHost.Controls.Clear();

    tabControl1.Dock = DockStyle.Fill;

    tabControl1.Margin = Padding.Empty;

    rightPanelHost.Controls.Add(tabControl1);

    rightPanelHost.MinimumSize = new Size(320, 0);

    mainLayout.Controls.Add(rightPanelHost, 2, 0);


    tableLayoutPanel1.Controls.Add(mainLayout, 0, 2);
}


private void ConfigureNavigation()
{
    navigationButtons.Clear();

    tableLayoutPanel2.Controls.Clear();

    tableLayoutPanel2.RowStyles.Clear();

    tableLayoutPanel2.ColumnStyles.Clear();

    tableLayoutPanel2.ColumnCount = 1;

    tableLayoutPanel2.RowCount = 0;

    tableLayoutPanel2.ColumnStyles.Add(new ColumnStyle(SizeType.Percent,
100F));


    var sections = new[]
    {
        "Flexível - cantoneira",
        "Flexível - chapa extremidade",
        "Flexível - chapa simples",
        "Rígida soldada",
    }

```

```
"Base rotulada",  
"Base engastada",  
"Mistas / anexos"  
};
```

```
float rowPercent = sections.Length > 0 ? 100f / sections.Length : 100f;
```

```
foreach (var section in sections)
```

```
{  
    var button = new Button  
    {  
        Text = section,  
        Dock = DockStyle.Fill,  
        FlatStyle = FlatStyle.Standard,  
        TextAlign = ContentAlignment.MiddleLeft,  
        Padding = new Padding(16, 10, 12, 10),  
        Tag = section,  
        Height = 45,  
        Cursor = Cursors.Hand  
    };  
    EngineeringStyle.StyleSidebarButton(button, false);  
    button.Click += OnNavigationClicked;  
  
    navigationButtons.Add(button);  
    tableLayoutPanel2.RowCount++;  
    tableLayoutPanel2.RowStyles.Add(new RowStyle(SizeType.Percent,  
rowPercent));  
    tableLayoutPanel2.Controls.Add(button, 0, tableLayoutPanel2.RowCount -  
1);
```

```
}  
}
```

```
private void ConfigureTabs()  
{  
    tabPage1.Controls.Clear();  
    tabPage2.Controls.Clear();  
    if (tabControl1.TabPages.Contains(tabPage3))  
    {  
        tabControl1.TabPages.Remove(tabPage3);  
    }  
  
    tabPage1.Text = "2D";  
    tabPage2.Text = "3D";  
  
    ConfigureTab2D();  
    ConfigureTab3D();  
}
```

```
private void ConfigureTab2D()  
{  
    tabPage1.Controls.Clear();  
  
    MoveInputsLayoutTo(tabPage1);  
  
    schematicPanel = Build2DViewsPanel();  
}
```



```
private void ConfigureTab3D()
{
    tabPage2.Controls.Clear();

    view3DHost = new Helix3DHost
    {
        Dock = DockStyle.Fill
    };

    var title = new Label
    {
        Dock = DockStyle.Top,
        Text = "Visualização 3D - ligação flexível",
        Height = 28,
        Font = new Font(Font.FontFamily, 14F, FontStyle.Bold)
    };

    var rightPanel = new Panel
    {
        Dock = DockStyle.Fill,
        Padding = new Padding(0)
    };

    view3DHost.Dock = DockStyle.Fill;
    rightPanel.Controls.Add(view3DHost);
    rightPanel.Controls.Add(title);

    view3DPanel = rightPanel;
}
```

```
private Control GetSharedInputsLayout()
{
    if (geometryInputsLayout == null)
    {
        geometryInputsLayout = BuildInputsLayout();
    }

    return geometryInputsLayout;
}

private void MoveInputsLayoutTo(TabPage targetTab)
{
    if (targetTab == null) return;

    var layout = GetSharedInputsLayout();
    if (layout.Parent == targetTab)
    {
        return;
    }

    if (layout.Parent != null)
    {
        layout.Parent.Controls.Remove(layout);
    }

    targetTab.Controls.Add(layout);
    layout.Dock = DockStyle.Fill;
}
```

```
}
```

```
private void EnsureInputsLayoutOnSelectedTab()
{
    MoveInputsLayoutTo(tabControl1?.SelectedTab ?? tabPage1);
}
```

```
private Control BuildInputsLayout()
{
    var layout = new TableLayoutPanel
    {
        Dock = DockStyle.Fill,
        ColumnCount = 1,
        AutoScroll = true,
        Padding = new Padding(12)
    };
    layout.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100F));

    layout.RowStyles.Add(new RowStyle());
    layout.Controls.Add(BuildBeamGroup(), 0, layout.RowCount);
    layout.RowCount++;
    layout.RowStyles.Add(new RowStyle());
    layout.Controls.Add(BuildColumnGroup(), 0, layout.RowCount);
    layout.RowCount++;
    layout.RowStyles.Add(new RowStyle());
    layout.Controls.Add(BuildPlateGroup(), 0, layout.RowCount);
    layout.RowCount++;
    layout.RowStyles.Add(new RowStyle());
```

```

        layout.Controls.Add(BuildBoltGroup(), 0, layout.RowCount);

        layout.RowCount++;

        layout.RowStyles.Add(new RowStyle());

        layout.Controls.Add(BuildWeldGroup(), 0, layout.RowCount);


        return layout;
    }

```

```

private Control BuildBeamGroup()
{
    var group = new GroupBox
    {
        Text = "Viga (perfil ou entrada manual)",
        Dock = DockStyle.Top,
        Padding = new Padding(15, 30, 15, 15),
        AutoSize = true,
        AutoSizeMode = AutoSizeMode.GrowAndShrink,
        BackColor = Color.White
    };

    group.Font = new Font(Font.FontFamily, 13F, FontStyle.Bold);


    var table = CreateFormTable();

    table.Font = this.Font;

    var row = 0;


    beamProfileCombo = new ComboBox { Dock = DockStyle.Fill,
        DropDownStyle = ComboBoxStyle.DropDownList };

    EngineeringStyle.StyleInputControl(beamProfileCombo);

```

```

        PopulateProfileCombo(beamProfileCombo);

        beamProfileCombo.SelectedIndexChanged +=
BeamProfileCombo_OnSelectedIndexChanged;

        var perfilLabel = new Label { Text = "Perfil", TextAlign =
ContentAlignment.MiddleRight, Dock = DockStyle.Fill, Anchor = AnchorStyles.Left
| AnchorStyles.Right };

        EngineeringStyle.StyleInputControl(perfilLabel);

        table.Controls.Add(perfilLabel, 0, row);

        table.Controls.Add(beamProfileCombo, 1, row);

        table.SetColumnSpan(beamProfileCombo, 2);

        row++;

        beamHeightBox = CreateNumericBox("BeamHeight",
OnGeometryTextChanged);

        AddInputRow(table, ref row, "Altura h_w", beamHeightBox);

        beamFlangeBox = CreateNumericBox("BeamFlangeThickness",
OnGeometryTextChanged);

        AddInputRow(table, ref row, "Espessura mesa t_f", beamFlangeBox);

        beamWebBox = CreateNumericBox("BeamWebThickness",
OnGeometryTextChanged);

        AddInputRow(table, ref row, "Espessura alma t_w", beamWebBox);

        beamFlangeWidthBox = CreateNumericBox("BeamFlangeWidth",
OnGeometryTextChanged);

        AddInputRow(table, ref row, "Largura mesa bf_w", beamFlangeWidthBox);

        group.Controls.Add(table);

```

```
    return group;
}
```

```
private Control BuildColumnGroup()
{
    var group = new GroupBox
    {
        Text = "Pilar (perfil ou entrada manual)",
        Dock = DockStyle.Top,
        Padding = new Padding(15, 30, 15, 15),
        AutoSize = true,
        AutoSizeMode = AutoSizeMode.GrowAndShrink,
        BackColor = Color.White
    };

    group.Font = new Font(Font.FontFamily, 13F, FontStyle.Bold);

    var table = CreateFormTable();

    table.Font = this.Font;

    var row = 0;

    columnProfileCombo = new ComboBox { Dock = DockStyle.Fill,
    DropDownStyle = ComboBoxStyle.DropDownList };

    EngineeringStyle.StyleInputControl(columnProfileCombo);

    PopulateProfileCombo(columnProfileCombo);

    columnProfileCombo.SelectedIndexChanged +=
    ColumnProfileCombo_OnSelectedIndexChanged;
```

```
var perfilLabel = new Label { Text = "Perfil", TextAlign =
ContentAlignment.MiddleRight, Dock = DockStyle.Fill, Anchor = AnchorStyles.Left
| AnchorStyles.Right };
```

```
EngineeringStyle.StyleInputControl(perfilLabel);

table.Controls.Add(perfilLabel, 0, row);

table.Controls.Add(columnProfileCombo, 1, row);

table.SetColumnSpan(columnProfileCombo, 2);

row++;
```

```
columnHeightBox = CreateNumericBox("ColumnDepth",
OnGeometryTextChanged);
```

```
AddInputRow(table, ref row, "Altura h_w,c", columnHeightBox);
```

```
columnFlangeBox = CreateNumericBox("ColumnFlangeThickness",
OnGeometryTextChanged);
```

```
AddInputRow(table, ref row, "Espessura mesa t_f,c", columnFlangeBox);
```

```
columnWebBox = CreateNumericBox("ColumnWebThickness",
OnGeometryTextChanged);
```

```
AddInputRow(table, ref row, "Espessura alma t_w,c", columnWebBox);
```

```
columnFlangeWidthBox = CreateNumericBox("ColumnWidth",
OnGeometryTextChanged);
```

```
AddInputRow(table, ref row, "Largura bf_c", columnFlangeWidthBox);
```

```
group.Controls.Add(table);
```

```
return group;
```

```
}
```

```
private Control BuildPlateGroup()
```

```

{
    var group = new GroupBox
    {
        Text = "Placa e bordas",
        Dock = DockStyle.Top,
        Padding = new Padding(15, 30, 15, 15),
        AutoSize = true,
        AutoSizeMode = AutoSizeMode.GrowAndShrink,
        BackColor = Color.White
    };

    var table = CreateFormTable();
    group.Font = new Font(Font.FontFamily, 13F, FontStyle.Bold);
    table.Font = this.Font;
    var row = 0;

    plateThicknessBox = CreateNumericBox("PlateThickness",
    OnGeometryTextChanged);

    AddInputRow(table, ref row, "Espessura t_p", plateThicknessBox);

    plateWidthBox = CreateNumericBox("PlateWidth",
    OnGeometryTextChanged);

    AddInputRow(table, ref row, "Largura l_p", plateWidthBox);

    plateEdgeHorizontalBox = CreateNumericBox("PlateEdgeHorizontal",
    OnGeometryTextChanged, true);

    AddInputRow(table, ref row, "Borda lateral e_h", plateEdgeHorizontalBox);

```



```

        plateEdgeTopBox = CreateNumericBox("PlateEdgeTop",
OnGeometryTextChanged, true);

        AddInputRow(table, ref row, "Borda superior e_t,p", plateEdgeTopBox);


        plateEdgeBottomBox = CreateNumericBox("PlateEdgeBottom",
OnGeometryTextChanged, true);

        AddInputRow(table, ref row, "Borda inferior e_b,p", plateEdgeBottomBox);


        group.Controls.Add(table);

        return group;
    }


    private Control BuildBoltGroup()
    {
        var group = new GroupBox
        {
            Text = "Parafusos (ligação flexível)",
            Dock = DockStyle.Top,
            Padding = new Padding(15, 30, 15, 15),
            AutoSize = true,
            AutoSizeMode = AutoSizeMode.GrowAndShrink,
            BackColor = Color.White
        };


        var table = CreateFormTable();

        group.Font = new Font(Font.FontFamily, 13F, FontStyle.Bold);

        table.Font = this.Font;

        var row = 0;

```

```

        boltDiameterBox = CreateNumericBox("BoltDiameter",
OnGeometryTextChanged);

        AddInputRow(table, ref row, "Diâmetro d_b", boltDiameterBox);


        boltCountBox = CreateNumericBox("BoltCount", OnBoltCountChanged);

        AddInputRow(table, ref row, "Nº linhas n", boltCountBox, "un");


        boltEdgeTopBox = CreateNumericBox("BoltEdgeTop",
OnGeometryTextChanged);

        AddInputRow(table, ref row, "Borda superior e_t", boltEdgeTopBox);


        boltSpacingBox = CreateNumericBox("BoltSpacing",
OnGeometryTextChanged);

        AddInputRow(table, ref row, "Passo vertical p_v", boltSpacingBox);


        boltGaugeBox = CreateNumericBox("BoltGauge", OnGeometryTextChanged);

        AddInputRow(table, ref row, "Gabarito g_h", boltGaugeBox);


        group.Controls.Add(table);

        return group;
    }


    private Control BuildWeldGroup()
    {
        var group = new GroupBox
        {
            Text = "Solda / geral",

            Dock = DockStyle.Top,

            Padding = new Padding(15, 30, 15, 15),

```

```
        AutoSize = true,  
        AutoSizeMode = AutoSizeMode.GrowAndShrink,  
        BackColor = Color.White  
    };
```

```
var table = CreateFormTable();  
group.Font = new Font(Font.FontFamily, 13F, FontStyle.Bold);  
table.Font = this.Font;  
var row = 0;
```

```
weldSizeBox = CreateNumericBox("WeldSize", OnGeometryTextChanged);  
AddInputRow(table, ref row, "Perna solda s_w", weldSizeBox);
```

```
group.Controls.Add(table);  
return group;  
}
```

```
private Panel Build2DViewsPanel()  
{  
    var panel = new Panel  
    {  
        Dock = DockStyle.Fill,  
        BackColor = EngineeringStyle.BackgroundColor,  
        Padding = new Padding(8)  
    };  
}
```

```
var layout = new TableLayoutPanel  
{
```

```

        Dock = DockStyle.Fill,

        ColumnCount = 1,

        RowCount = 3

    };

    layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));
    layout.RowStyles.Add(new RowStyle(SizeType.Percent, 50F));
    layout.RowStyles.Add(new RowStyle(SizeType.Percent, 50F));

    var title = new Label
    {
        Dock = DockStyle.Top,
        Text = "Visualização 2D (vista lateral e frontal) - ligação flexível",
        Font = new Font(Font.FontFamily, 14F, FontStyle.Bold),
        Height = 26,
        TextAlign = ContentAlignment.MiddleLeft
    };

    layout.Controls.Add(title, 0, 0);


    sideViewPanel = new DoubleBufferedPanel
    {
        Dock = DockStyle.Fill,
        BackColor = Color.White,
        MinimumSize = new Size(200, 150)
    };

    sideViewPanel.Paint += DrawSideView;


    frontViewPanel = new DoubleBufferedPanel
    {

```

```

        Dock = DockStyle.Fill,
        BackColor = Color.White,
        MinimumSize = new Size(200, 150)
    };
    frontViewPanel.Paint += DrawFrontView;

    layout.Controls.Add(sideViewPanel, 0, 1);
    layout.Controls.Add(frontViewPanel, 0, 2);

    panel.Controls.Add(layout);
    return panel;
}

private static TableLayoutPanel CreateFormTable()
{
    var table = new TableLayoutPanel
    {
        Dock = DockStyle.Fill,
        ColumnCount = 3,
        AutoSize = true,
        AutoSizeMode = AutoSizeMode.GrowAndShrink,
        Padding = new Padding(6)
    };
    table.ColumnStyles.Add(new ColumnStyle(SizeType.AutoSize));
    table.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100F));
    table.ColumnStyles.Add(new ColumnStyle(SizeType.AutoSize));
    return table;
}

```

```
private TextBox CreateNumericBox(string tag, EventHandler handler, bool  
readOnly = false)
```

```
{  
    var box = new TextBox  
    {  
        Dock = DockStyle.Fill,  
        TextAlign = HorizontalAlignment.Right,  
        ReadOnly = readOnly  
    };  
    box.Tag = tag;  
    box.TextChanged += handler;  
    EngineeringStyle.StyleInputControl(box);  
    return box;  
}
```

```
private void AddInputRow(TableLayoutPanel table, ref int row, string labelText,  
TextBox box, string unit = "mm")
```

```
{  
    table.RowStyles.Add(new RowStyle(SizeType.Absolute, 40f));  
  
    var label = new Label  
    {  
        Text = labelText,  
        TextAlign = ContentAlignment.MiddleRight,  
        Dock = DockStyle.Fill,  
        Anchor = AnchorStyles.Left | AnchorStyles.Right,  
        Padding = new Padding(0, 10, 4, 0)  
    };
```

```
EngineeringStyle.StyleInputControl(label);
```

```
var unitLabel = new Label  
{  
    Text = unit,  
    TextAlign = ContentAlignment.MiddleLeft,  
    Dock = DockStyle.Fill,  
    Anchor = AnchorStyles.Left | AnchorStyles.Right,  
    Padding = new Padding(4, 10, 0, 0)  
};  
EngineeringStyle.StyleInputControl(unitLabel);
```

```
EngineeringStyle.StyleInputControl(box);
```

```
table.Controls.Add(label, 0, row);  
table.Controls.Add(box, 1, row);  
table.Controls.Add(unitLabel, 2, row);  
row++;  
}
```

```
private void ConfigureFooter()  
{  
    if (tableLayoutPanel1.RowStyles.Count > 3)  
    {  
        tableLayoutPanel1.RowStyles[3] = new RowStyle(SizeType.AutoSize);  
    }  
}
```

```
var existingFooter = tableLayoutPanel1.GetControlFromPosition(0, 3);
```

```
if (existingFooter != null)
{
    tableLayoutPanel1.Controls.Remove(existingFooter);
}

var footerPanel = new TableLayoutPanel
{
    Dock = DockStyle.Fill,
    ColumnCount = 3,
    RowCount = 1,
    Padding = new Padding(6)
};

footerPanel.ColumnStyles.Add(new ColumnStyle(SizeType.AutoSize));
footerPanel.ColumnStyles.Add(new ColumnStyle(SizeType.AutoSize));
footerPanel.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100F));
footerPanel.RowStyles.Add(new RowStyle(SizeType.Percent, 100F));

calculateButton = new Button
{
    Text = "Calcular",
    AutoSize = true,
    Dock = DockStyle.Left,
    Padding = new Padding(10, 6, 10, 6)
};

calculateButton.Click += OnCalculateClicked;

generateReportButton = new Button
{
```



```

        Text = "Gerar Relatório",

        AutoSize = true,

        Dock = DockStyle.Left,

        Padding = new Padding(10, 6, 10, 6)

    };

    generateReportButton.Click += OnGenerateReportClicked;

    if (reportBox == null)
    {
        reportBox = new RichTextBox();
    }

    statusLabel = new ToolStripStatusLabel { Text = "Pronto" };
    statusStrip = new StatusStrip
    {
        Dock = DockStyle.Fill,
        SizingGrip = false
    };

    statusStrip.Items.Add(statusLabel);

    footerPanel.Controls.Add(calculateButton, 0, 0);
    footerPanel.Controls.Add(generateReportButton, 1, 0);
    footerPanel.Controls.Add(statusStrip, 2, 0);

    tableLayoutPanel1.Controls.Add(footerPanel, 0, 3);
}

private void CreateOverlay()

```

```

{
    if (overlayPanel != null) return;

    overlayPanel = new Panel
    {
        Anchor = AnchorStyles.Top | AnchorStyles.Left,
        BackColor = Color.White,
        Visible = false,
        BorderStyle = BorderStyle.FixedSingle,
        Size = new Size(1100, 700)
    };

    overlayRestoreBounds = new Rectangle(new Point(80, 80),
overlayPanel.Size);

    overlayPanel.Location = overlayRestoreBounds.Location;

    try
    {
        captionIconFont = new Font("Segoe UI Symbol", 9.5f, FontStyle.Regular);
    }
    catch
    {
        captionIconFont = new Font(Font, FontStyle.Regular);
    }

    overlayHeader = new TableLayoutPanel
    {
        Dock = DockStyle.Top,
        Height = 34,

```

```

        ColumnCount = 3,

        RowCount = 1,

        Padding = new Padding(8, 4, 8, 4)
    };

    overlayHeader.ColumnStyles.Add(new ColumnStyle(SizeType.Percent,
100F));

    overlayHeader.ColumnStyles.Add(new ColumnStyle(SizeType.AutoSize));

    overlayHeader.ColumnStyles.Add(new ColumnStyle(SizeType.AutoSize));


    overlayTitle = new Label
    {
        Text = "",
        AutoSize = true,
        Dock = DockStyle.Fill,
        Font = new Font(Font, FontStyle.Bold),
        TextAlign = ContentAlignment.MiddleLeft
    };

    overlayMaxButton = new Button
    {
        Text = "□",
        AutoSize = false,
        Dock = DockStyle.Fill,
        Padding = Padding.Empty,
        MinimumSize = new Size(40, 26),
        Margin = new Padding(4, 0, 4, 0),
        Font = captionIconFont,
        TextAlign = ContentAlignment.MiddleCenter,
        FlatStyle = FlatStyle.Standard
    };

```

```
};  
overlayMaxButton.Click += (s, e) => ToggleOverlaySize();
```

```
overlayCloseButton = new Button  
{  
    Text = "x",  
    AutoSize = false,  
    Dock = DockStyle.Fill,  
    Padding = Padding.Empty,  
    MinimumSize = new Size(40, 26),  
    Margin = new Padding(0),  
    Font = captionIconFont,  
    TextAlign = ContentAlignment.MiddleCenter,  
    FlatStyle = FlatStyle.Standard  
};  
overlayCloseButton.Click += (s, e) => HideOverlay();
```

```
overlayHeader.Controls.Add(overlayTitle, 0, 0);  
overlayHeader.Controls.Add(overlayMaxButton, 1, 0);  
overlayHeader.Controls.Add(overlayCloseButton, 2, 0);
```

```
overlayContent = new TableLayoutPanel  
{  
    Dock = DockStyle.Fill,  
    ColumnCount = 2,  
    RowCount = 1,  
    Padding = new Padding(12)  
};
```

```
overlayContent.ColumnStyles.Add(new ColumnStyle(SizeType.Percent,
50F));
```

```
overlayContent.ColumnStyles.Add(new ColumnStyle(SizeType.Percent,
50F));
```

```
overlayContent.RowStyles.Add(new RowStyle(SizeType.Percent, 100F));
```

```
overlayPlaceholder = new Panel
```

```
{
```

```
    Dock = DockStyle.Fill,
```

```
    BackColor = Color.White,
```

```
    Padding = new Padding(16)
```

```
};
```

```
overlayPanel.Controls.Add(overlayContent);
```

```
overlayPanel.Controls.Add(overlayHeader);
```

```
Controls.Add(overlayPanel);
```

```
overlayPanel.BringToFront();
```

```
overlayPanel.Visible = false;
```

```
overlayHeader.MouseDown += OverlayHeader_MouseDown;
```

```
overlayHeader.MouseMove += OverlayHeader_MouseMove;
```

```
overlayHeader.MouseUp += OverlayHeader_MouseUp;
```

```
overlayTitle.MouseDown += OverlayHeader_MouseDown;
```

```
overlayTitle.MouseMove += OverlayHeader_MouseMove;
```

```
overlayTitle.MouseUp += OverlayHeader_MouseUp;
```

```
}
```

```
private Panel BuildPlaceholder(string title, string message)
```

```
{  
    var panel = new Panel  
    {  
        Dock = DockStyle.Fill,  
        BackColor = Color.White,  
        Padding = new Padding(16)  
    };  
  
    var titleLabel = new Label  
    {  
        Dock = DockStyle.Top,  
        TextAlign = ContentAlignment.MiddleLeft,  
        Text = title,  
        Font = new Font(Font, FontStyle.Bold),  
        Height = 28  
    };  
  
    var messageLabel = new Label  
    {  
        Dock = DockStyle.Fill,  
        TextAlign = ContentAlignment.TopLeft,  
        Text = message,  
        Font = new Font(Font.FontFamily, 11f, FontStyle.Regular)  
    };  
  
    panel.Controls.Add(messageLabel);  
    panel.Controls.Add(titleLabel);  
    return panel;
```

```
}
```

```
private void ShowGeometryOverlay()
{
    if (overlayPanel == null) return;
    overlayMode = "Geometry";
    overlayTitle.Text = "Geometria";

    if (!linkTypeSelected)
    {
        ShowStatus("Selecione um tipo de ligação na coluna à esquerda antes de
abrir a Geometria.", true);
        return;
    }

    if (tabControl1.Parent != overlayContent)
    {
        if (mainLayout.Controls.Contains(tabControl1))
        {
            mainLayout.Controls.Remove(tabControl1);
        }
        if (mainLayout.Controls.Contains(rightPanelHost))
        {
            mainLayout.Controls.Remove(rightPanelHost);
        }

        overlayContent.Controls.Clear();
        overlayContent.Controls.Add(tabControl1, 0, 0);
    }
}
```

```
        overlayContent.Controls.Add(rightPanelHost, 1, 0);  
    }  
}
```

```
overlayPanel.Bounds = overlayRestoreBounds;  
overlayPanel.Dock = DockStyle.None;  
overlayMaximized = false;  
overlayMaxButton.Text = "□";
```

```
overlayPanel.Visible = true;  
overlayPanel.BringToFront();  
UpdateRightPanelContent();  
}
```

```
private void ShowGenericOverlay(string title, string message)  
{  
    if (overlayPanel == null) return;  
    overlayMode = title;  
    overlayTitle.Text = title;  
  
    if (!mainLayout.Controls.Contains(tabControl1))  
    {  
        overlayContent.Controls.Remove(tabControl1);  
        mainLayout.Controls.Add(tabControl1, 1, 0);  
    }  
    if (!mainLayout.Controls.Contains(rightPanelHost))  
    {  
        overlayContent.Controls.Remove(rightPanelHost);  
        mainLayout.Controls.Add(rightPanelHost, 2, 0);  
    }  
}
```



```
}
```

```
overlayContent.Controls.Clear();  
overlayPlaceholder.Controls.Clear();  
var placeholder = BuildPlaceholder(title, message);  
overlayPlaceholder.Controls.Add(placeholder);  
overlayContent.Controls.Add(overlayPlaceholder, 0, 0);  
overlayContent.SetColumnSpan(overlayPlaceholder, 2);
```

```
overlayPanel.Bounds = overlayRestoreBounds;  
overlayPanel.Dock = DockStyle.None;  
overlayMaximized = false;  
overlayMaxButton.Text = "□";
```

```
overlayPanel.Visible = true;  
overlayPanel.BringToFront();
```

```
}
```

```
private void HideOverlay()  
{  
    if (overlayPanel == null) return;  
  
    if (overlayMode == "Geometry")  
    {  
        if (overlayContent.Controls.Contains(tabControl1))  
        {  
            overlayContent.Controls.Remove(tabControl1);  
            mainLayout.Controls.Add(tabControl1, 1, 0);  
        }  
    }  
}
```

```
    }  
    if (overlayContent.Controls.Contains(rightPanelHost))  
    {  
        overlayContent.Controls.Remove(rightPanelHost);  
        mainLayout.Controls.Add(rightPanelHost, 2, 0);  
    }  
}
```

```
overlayPanel.Visible = false;  
overlayMode = null;  
UpdateRightPanelContent();  
}
```

```
private void ToggleOverlaySize()  
{  
    if (overlayPanel == null) return;  
  
    if (overlayMaximized)  
    {  
        overlayPanel.Dock = DockStyle.None;  
        overlayPanel.Bounds = overlayRestoreBounds;  
        overlayMaxButton.Text = "□";  
        overlayMaximized = false;  
    }  
    else  
    {  
        overlayRestoreBounds = overlayPanel.Bounds;  
        overlayPanel.Dock = DockStyle.Fill;
```

```
        overlayMaxButton.Text = "□";  
        overlayMaximized = true;  
    }  
}
```

```
private void OverlayHeader_MouseDown(object sender, MouseEventArgs e)  
{  
    if (overlayMaximized) return;  
    if (e.Button == MouseButtons.Left)  
    {  
        overlayDragging = true;  
        overlayDragOffset = e.Location;  
    }  
}
```

```
private void OverlayHeader_MouseMove(object sender, MouseEventArgs e)  
{  
    if (overlayDragging && !overlayMaximized)  
    {  
        var newLocation = overlayPanel.Location;  
        newLocation.Offset(e.X - overlayDragOffset.X, e.Y - overlayDragOffset.Y);  
        overlayPanel.Location = newLocation;  
    }  
}
```

```
private void OverlayHeader_MouseUp(object sender, MouseEventArgs e)  
{  
    overlayDragging = false;
```

```
}
```

```
private void OnNavigationClicked(object sender, EventArgs e)
```

```
{
```

```
    if (sender is Button button)
```

```
    {
```

```
        SetActiveNavigationButton(button);
```

```
        tabControl1.SelectedTab = tabPage1;
```

```
        ShowStatus($"Etapla seleccionada: {button.Text}", false);
```

```
        linkTypeSelected = true;
```

```
        UpdateRightPanelContent();
```

```
    }
```

```
}
```

```
private void OnTabChanged(object sender, EventArgs e)
```

```
{
```

```
    EnsureInputsLayoutOnSelectedTab();
```

```
    UpdateRightPanelContent();
```

```
}
```

```
private void SetActiveNavigationButton(Button selected)
```

```
{
```

```
    foreach (var button in navigationButtons)
```

```
    {
```

```
        if (button == selected)
```

```
        {
```

```
            EngineeringStyle.StyleSidebarButton(button, true);
```

```
        }
```

```

        else
        {
            EngineeringStyle.StyleSidebarButton(button, false);
        }
    }
}

```

```

private void UpdateRightPanelContent()

```

```

{
    if (rightPanelHost == null)
    {
        return;
    }

```

```

    rightPanelHost.Controls.Clear();

```

```

    if (tabControl1.SelectedTab == tabPage1 && schematicPanel != null)
    {
        rightPanelHost.Controls.Add(schematicPanel);
    }

    else if (tabControl1.SelectedTab == tabPage2 && view3DPanel != null)
    {
        rightPanelHost.Controls.Add(view3DPanel);
    }

    // Aba de relatório removida na geometria; mantém 2D/3D.
}

```

```

private void OnCalculateClicked(object sender, EventArgs e)

```

```

{
    if (!TrySyncGeometryFromInputs())
    {
        ShowStatus("Valores de geometria inválidos. Revise os campos.", true);
        return;
    }

    data.geometry.hx = geometryParams.BeamHeight * 1000.0;
    data.geometry.hy = geometryParams.BeamThickness * 1000.0;

    ShowStatus($"Geometria (flexível) atualizada: h_w = {data.geometry.hx:0.##} mm, bf_w = {data.geometry.hy:0.##} mm, n = {geometryParams.BoltCount} Ø{geometryParams.BoltDiameter * 1000:0.##} mm.", false);

    RefreshGeometryViews();
}

private void OnGenerateReportClicked(object sender, EventArgs e)
{
    var sb = new StringBuilder();

    sb.AppendLine("Memorial de Cálculo - Geometria (flexível)");
    sb.AppendLine("-----");

    sb.AppendLine($"Viga: h_w = {geometryParams.BeamHeight * 1000:0.##} mm, t_f = {geometryParams.BeamFlangeThickness * 1000:0.##} mm, t_w = {geometryParams.BeamWebThickness * 1000:0.##} mm, bf_w = {geometryParams.BeamThickness * 1000:0.##} mm");

    sb.AppendLine($"Pilar: h_w,c = {geometryParams.ColumnDepth * 1000:0.##} mm, t_f,c = {geometryParams.ColumnFlangeThickness * 1000:0.##} mm, t_w,c = {geometryParams.ColumnWebThickness * 1000:0.##} mm, bf_c = {geometryParams.ColumnWidth * 1000:0.##} mm");

    sb.AppendLine($"Placa: t_p = {geometryParams.PlateThickness * 1000:0.##} mm, l_p = {geometryParams.PlateWidth * 1000:0.##} mm, e_h =

```

```

{geometryParams.PlateEdgeHorizontal * 1000:0.##} mm, e_t =
{geometryParams.PlateEdgeTop * 1000:0.##} mm, e_b =
{geometryParams.PlateEdgeBottom * 1000:0.##} mm");

    sb.AppendLine($"Parafusos: n = {geometryParams.BoltCount}, p_v =
{geometryParams.BoltSpacing * 1000:0.##} mm, g_h = {geometryParams.BoltGauge
* 1000:0.##} mm, d_b = {geometryParams.BoltDiameter * 1000:0.##} mm");

    sb.AppendLine($"Solda (filete): s_w = {geometryParams.WeldSize * 1000:0.##}
mm");

    sb.AppendLine($"Data/Hora: {DateTime.Now}");

    if (reportBox != null)
    {
        reportBox.Text = sb.ToString();
    }

    ShowStatus("Relatório gerado na aba Relatório.", false);
}

private void LoadGeometryToInputs()
{
    geometryInputsUpdating = true;

    if (data?.geometry != null)
    {
        if (data.geometry.hx > 0) geometryParams.BeamHeight = data.geometry.hx
/ 1000.0;

        if (data.geometry.hy > 0) geometryParams.BeamThickness =
data.geometry.hy / 1000.0;
    }
}

```

```

PushGeometryToInputs();

geometryInputsUpdating = false;


SelectProfileByName(beamProfileCombo, "W 250x22,3");
SelectProfileByName(columnProfileCombo, "W 310x52,0");


UpdateDerivedGeometry();

RefreshGeometryViews();
}


private void PushGeometryToInputs()
{
    SetText(beamHeightBox, geometryParams.BeamHeight);
    SetText(beamFlangeBox, geometryParams.BeamFlangeThickness);
    SetText(beamWebBox, geometryParams.BeamWebThickness);
    SetText(beamFlangeWidthBox, geometryParams.BeamThickness);


    SetText(columnHeightBox, geometryParams.ColumnDepth);
    SetText(columnFlangeBox, geometryParams.ColumnFlangeThickness);
    SetText(columnWebBox, geometryParams.ColumnWebThickness);
    SetText(columnFlangeWidthBox, geometryParams.ColumnWidth);


    SetText(plateThicknessBox, geometryParams.PlateThickness);
    SetText(plateWidthBox, geometryParams.PlateWidth);
    SetText(plateEdgeHorizontalBox, geometryParams.PlateEdgeHorizontal);
    SetText(plateEdgeTopBox, geometryParams.PlateEdgeTop);
    SetText(plateEdgeBottomBox, geometryParams.PlateEdgeBottom);

```



```

        boltCountBox.Text =
geometryParams.BoltCount.ToString(CultureInfo.InvariantCulture);

        SetText(boltSpacingBox, geometryParams.BoltSpacing);

        SetText(boltEdgeTopBox, geometryParams.BoltEdgeTop);

        SetText(boltGaugeBox, geometryParams.BoltGauge);

        SetText(boltDiameterBox, geometryParams.BoltDiameter);


        SetText(weldSizeBox, geometryParams.WeldSize);
    }

    private void UpdateDerivedGeometry(bool pushToInputs = true)
    {
        geometryParams.BeamLength = Math.Max(0.1, 0.75 *
geometryParams.BeamHeight);

        geometryParams.PlateHeight = geometryParams.BeamHeight;

        var eH = (geometryParams.PlateWidth - geometryParams.BoltGauge) / 2.0;

        var eTop = geometryParams.BoltEdgeTop -
geometryParams.BeamFlangeThickness;

        var pitchTotal = Math.Max(0, geometryParams.BoltCount - 1) *
geometryParams.BoltSpacing;

        var eBottom = geometryParams.BeamHeight - eTop - pitchTotal;

        geometryParams.PlateEdgeHorizontal = eH;

        geometryParams.PlateEdgeTop = eTop;

        geometryParams.PlateEdgeBottom = eBottom;

        if (pushToInputs)
        {

```

```
        geometryInputsUpdating = true;

        SetText(plateEdgeHorizontalBox, geometryParams.PlateEdgeHorizontal);

        SetText(plateEdgeTopBox, geometryParams.PlateEdgeTop);

        SetText(plateEdgeBottomBox, geometryParams.PlateEdgeBottom);

        geometryInputsUpdating = false;
    }
}
```

```
private void RefreshGeometryViews()
{
    sideViewPanel?.Invalidate();
    frontViewPanel?.Invalidate();
    UpdateHelixScene();
}
```

```
private void UpdateHelixScene()
{
    if (helixSceneBuilder == null || view3DHost == null || geometryParams == null)
    {
        return;
    }
```

```
    var scene = helixSceneBuilder.BuildScene(geometryParams);
    view3DHost.SetScene(scene);
}
```

```
private void OnGeometryTextChanged(object sender, EventArgs e)
{

```

```
if (geometryInputsUpdating || !(sender is TextBox box) || box.ReadOnly)
{
    return;
}
```

```
if (!TryParseDouble(box.Text, out var valueMm) || valueMm <= 0)
{
    return;
}
```

```
var valueMeters = valueMm / 1000.0;
switch (box.Tag as string)
{
    case "BeamHeight":
        geometryParams.BeamHeight = valueMeters;
        break;
    case "BeamFlangeThickness":
        geometryParams.BeamFlangeThickness = valueMeters;
        break;
    case "BeamWebThickness":
        geometryParams.BeamWebThickness = valueMeters;
        break;
    case "BeamFlangeWidth":
        geometryParams.BeamThickness = valueMeters;
        break;
    case "ColumnDepth":
        geometryParams.ColumnDepth = valueMeters;
        break;
}
```

```
case "ColumnFlangeThickness":  
    geometryParams.ColumnFlangeThickness = valueMeters;  
    break;  
case "ColumnWebThickness":  
    geometryParams.ColumnWebThickness = valueMeters;  
    break;  
case "ColumnWidth":  
    geometryParams.ColumnWidth = valueMeters;  
    geometryParams.ColumnWebWidth = valueMeters;  
    break;  
case "PlateThickness":  
    geometryParams.PlateThickness = valueMeters;  
    break;  
case "PlateWidth":  
    geometryParams.PlateWidth = valueMeters;  
    break;  
case "BoltSpacing":  
    geometryParams.BoltSpacing = valueMeters;  
    break;  
case "BoltEdgeTop":  
    geometryParams.BoltEdgeTop = valueMeters;  
    break;  
case "BoltGauge":  
    geometryParams.BoltGauge = valueMeters;  
    break;  
case "BoltDiameter":  
    geometryParams.BoltDiameter = valueMeters;  
    break;
```

```
        case "WeldSize":  
            geometryParams.WeldSize = valueMeters;  
            break;  
        default:  
            return;  
    }  
  
    UpdateDerivedGeometry();  
    RefreshGeometryViews();  
}
```

```
UpdateDerivedGeometry();  
RefreshGeometryViews();  
}
```

```
private void OnBoltCountChanged(object sender, EventArgs e)  
{  
    if (geometryInputsUpdating || !(sender is TextBox box))  
    {  
        return;  
    }  
}
```

```
        if (int.TryParse(box.Text, NumberStyles.Integer, CultureInfo.InvariantCulture,  
out var count) && count > 0)  
        {  
            geometryParams.BoltCount = count;  
            UpdateDerivedGeometry();  
            RefreshGeometryViews();  
        }  
    }  
}
```

```
private void BeamProfileCombo_OnSelectedIndexChanged(object sender,  
EventArgs e)
```

```
{  
    if (beamProfileCombo?.SelectedItem is ProfileDefinition profile)  
    {  
        ApplyBeamProfile(profile);  
    }  
    else  
    {  
        SetBeamInputsReadOnly(false);  
    }  
}
```

```
private void ColumnProfileCombo_OnSelectedIndexChanged(object sender,  
EventArgs e)
```

```
{  
    if (columnProfileCombo?.SelectedItem is ProfileDefinition profile)  
    {  
        ApplyColumnProfile(profile);  
    }  
    else  
    {  
        SetColumnInputsReadOnly(false);  
    }  
}
```

```
private void PopulateProfileCombo(ComboBox combo)
```

```
{  
    if (combo == null) return;  
    combo.Items.Clear();
```

```

        combo.Items.Add("-- Entrada manual --");

        foreach (var profile in ProfileLibrary)
        {
            combo.Items.Add(profile);
        }

        combo.SelectedIndex = 0;
    }

    private void SelectProfileByName(ComboBox combo, string name)
    {
        if (combo == null || string.IsNullOrEmpty(name))
        {
            return;
        }

        for (var i = 0; i < combo.Items.Count; i++)
        {
            if (combo.Items[i] is ProfileDefinition profile && profile.Name == name)
            {
                combo.SelectedIndex = i;
                return;
            }
        }
    }

    private void ApplyBeamProfile(ProfileDefinition profile)
    {
        geometryInputsUpdating = true;
    }

```

```

SetText(beamHeightBox, profile.DepthMm / 1000.0);

SetText(beamFlangeBox, profile.FlangeThicknessMm / 1000.0);

SetText(beamWebBox, profile.WebThicknessMm / 1000.0);

SetText(beamFlangeWidthBox, profile.FlangeWidthMm / 1000.0);

geometryInputsUpdating = false;

SetBeamInputsReadOnly(true);


geometryParams.BeamHeight = profile.DepthMm / 1000.0;

geometryParams.BeamFlangeThickness = profile.FlangeThicknessMm /
1000.0;

geometryParams.BeamWebThickness = profile.WebThicknessMm / 1000.0;

geometryParams.BeamThickness = profile.FlangeWidthMm / 1000.0;

UpdateDerivedGeometry();

RefreshGeometryViews();
}


private void ApplyColumnProfile(ProfileDefinition profile)
{
    geometryInputsUpdating = true;

    SetText(columnHeightBox, profile.DepthMm / 1000.0);

    SetText(columnFlangeBox, profile.FlangeThicknessMm / 1000.0);

    SetText(columnWebBox, profile.WebThicknessMm / 1000.0);

    SetText(columnFlangeWidthBox, profile.FlangeWidthMm / 1000.0);

    geometryInputsUpdating = false;

    SetColumnInputsReadOnly(true);


    geometryParams.ColumnDepth = profile.DepthMm / 1000.0;

    geometryParams.ColumnFlangeThickness = profile.FlangeThicknessMm /
1000.0;

```



```

        geometryParams.ColumnWebThickness = profile.WebThicknessMm / 1000.0;
        geometryParams.ColumnWidth = profile.FlangeWidthMm / 1000.0;
        geometryParams.ColumnWebWidth = profile.FlangeWidthMm / 1000.0;
        UpdateDerivedGeometry();
        RefreshGeometryViews();
    }

    private void SetBeamInputsReadOnly(bool isReadOnly)
    {
        if (beamHeightBox != null) beamHeightBox.ReadOnly = isReadOnly;
        if (beamFlangeBox != null) beamFlangeBox.ReadOnly = isReadOnly;
        if (beamWebBox != null) beamWebBox.ReadOnly = isReadOnly;
        if (beamFlangeWidthBox != null) beamFlangeWidthBox.ReadOnly =
isReadOnly;
    }

    private void SetColumnInputsReadOnly(bool isReadOnly)
    {
        if (columnHeightBox != null) columnHeightBox.ReadOnly = isReadOnly;
        if (columnFlangeBox != null) columnFlangeBox.ReadOnly = isReadOnly;
        if (columnWebBox != null) columnWebBox.ReadOnly = isReadOnly;
        if (columnFlangeWidthBox != null) columnFlangeWidthBox.ReadOnly =
isReadOnly;
    }

    private bool TrySyncGeometryFromInputs()
    {
        var ok = true;

```

```
ok &= TryParseMillimeters(beamHeightBox, out var bh);  
ok &= TryParseMillimeters(beamFlangeBox, out var bft);  
ok &= TryParseMillimeters(beamWebBox, out var bwt);  
ok &= TryParseMillimeters(beamFlangeWidthBox, out var bfw);
```

```
ok &= TryParseMillimeters(columnHeightBox, out var ch);  
ok &= TryParseMillimeters(columnFlangeBox, out var cft);  
ok &= TryParseMillimeters(columnWebBox, out var cwt);  
ok &= TryParseMillimeters(columnFlangeWidthBox, out var cfw);
```

```
ok &= TryParseMillimeters(plateThicknessBox, out var pt);  
ok &= TryParseMillimeters(plateWidthBox, out var pw);  
ok &= TryParseMillimeters(boltSpacingBox, out var bs);  
ok &= TryParseMillimeters(boltEdgeTopBox, out var be);  
ok &= TryParseMillimeters(boltGaugeBox, out var bg);  
ok &= TryParseMillimeters(boltDiameterBox, out var bd);  
ok &= TryParseMillimeters(weldSizeBox, out var ws);
```

```
if (!int.TryParse(boltCountBox?.Text, NumberStyles.Integer,  
CultureInfo.InvariantCulture, out var boltCount) || boltCount <= 0)
```

```
{  
    ok = false;  
}
```

```
if (!ok)  
{  
    return false;  
}
```

```
geometryParams.BeamHeight = bh;
geometryParams.BeamFlangeThickness = bft;
geometryParams.BeamWebThickness = bwt;
geometryParams.BeamThickness = bfw;

geometryParams.ColumnDepth = ch;
geometryParams.ColumnFlangeThickness = cft;
geometryParams.ColumnWebThickness = cwt;
geometryParams.ColumnWidth = cfw;
geometryParams.ColumnWebWidth = cfw;

geometryParams.PlateThickness = pt;
geometryParams.PlateWidth = pw;

geometryParams.BoltCount = boltCount;
geometryParams.BoltSpacing = bs;
geometryParams.BoltEdgeTop = be;
geometryParams.BoltGauge = bg;
geometryParams.BoltDiameter = bd;

geometryParams.WeldSize = ws;

UpdateDerivedGeometry();
return true;
}

private bool TryParseMillimeters(TextBox box, out double meters)
```

```

{
    meters = 0;
    if (box == null)
    {
        return false;
    }

    if (TryParseDouble(box.Text, out var valueMm) && valueMm > 0)
    {
        meters = valueMm / 1000.0;
        return true;
    }

    return false;
}

private bool TryParseDouble(string input, out double value)
{
    if (double.TryParse(input, NumberStyles.Float, CultureInfo.CurrentCulture,
out value))
    {
        return true;
    }

    return double.TryParse(input, NumberStyles.Float,
CultureInfo.InvariantCulture, out value);
}

private void SetText(TextBox box, double valueMeters)

```

```
{  
    if (box == null) return;  
    box.Text = (valueMeters * 1000.0).ToString("0.0",  
CultureInfo.InvariantCulture);  
}
```

```
private void DrawSideView(object sender, PaintEventArgs e)  
{  
    if (sideViewPanel == null) return;  
  
    var g = e.Graphics;  
    g.SmoothingMode = SmoothingMode.AntiAlias;  
  
    var width = sideViewPanel.ClientSize.Width;  
    var height = sideViewPanel.ClientSize.Height;  
    if (width <= 0 || height <= 0)  
    {  
        return;  
    }  
  
    const double margin = 60;  
    var p = geometryParams;  
    var columnHeightMm = Math.Max(p.ColumnDepth * 1000.0, 300);  
    var columnWidthMm = Math.Max(p.ColumnWidth * 1000.0, 150);  
    var beamLengthMm = Math.Max(p.BeamLength * 1000.0, 400);  
    var plateThicknessMm = Math.Max(p.PlateThickness * 1000.0, 10);  
    var plateHeightMm = Math.Max(p.PlateHeight * 1000.0, 200);  
    var beamHeightMm = Math.Max((p.BeamHeight + 2 *  
p.BeamFlangeThickness) * 1000.0, 200);
```

```

var totalWidthMm = columnWidthMm + beamLengthMm + 200;

var totalHeightMm = columnHeightMm + 150;

var scale = (float)Math.Min((width - margin) / totalWidthMm, (height - margin)
/ totalHeightMm);

if (float.IsNaN(scale) || scale <= 0) scale = 0.3f;

var columnWidthPx = (float)(columnWidthMm * scale);
var columnHeightPx = (float)(columnHeightMm * scale);
var columnLeft = (float)(margin / 2);
var columnTop = (float)((height - columnHeightPx) / 2);

using (var columnBrush = new SolidBrush(Color.FromArgb(180, 190, 205)))
{
    g.FillRectangle(columnBrush, columnLeft, columnTop, columnWidthPx,
columnHeightPx);
}

g.DrawRectangle(Pens.SlateGray, columnLeft, columnTop, columnWidthPx,
columnHeightPx);

var plateHeightPx = (float)(plateHeightMm * scale);
var plateThicknessPx = (float)(plateThicknessMm * scale);
var plateTop = columnTop + (columnHeightPx - plateHeightPx) / 2f;
var plateLeft = columnLeft + columnWidthPx - plateThicknessPx;

g.FillRectangle(Brushes.Gainsboro, plateLeft, plateTop, plateThicknessPx,
plateHeightPx);

g.DrawRectangle(Pens.Gray, plateLeft, plateTop, plateThicknessPx,
plateHeightPx);

```

```

var beamHeightPx = (float)(beamHeightMm * scale);

var beamThicknessPx = Math.Max((float)(p.BeamThickness * 1000.0 * scale),
plateThicknessPx * 5f);

var beamLeft = plateLeft + plateThicknessPx - 2;

var beamTop = columnTop + (columnHeightPx - beamHeightPx) / 2f;

var beamWidthPx = Math.Max((float)(beamLengthMm * scale), 120f);


var beamRect = new RectangleF(beamLeft, beamTop, beamWidthPx,
beamHeightPx);

g.FillRectangle(Brushes.WhiteSmoke, beamRect);

g.DrawRectangle(Pens.DimGray, beamRect.X, beamRect.Y,
beamRect.Width, beamRect.Height);


var boltRadius = Math.Max((float)(p.BoltDiameter * 1000.0 * scale * 0.2), 4f);
var boltSpacingPx = Math.Max((float)(p.BoltSpacing * 1000.0 * scale), 30f);
var firstBoltOffsetPx = Math.Max((float)(p.BoltEdgeTop * 1000.0 * scale), 30f);
var boltCenterX = plateLeft + plateThicknessPx / 2f;
for (var i = 0; i < p.BoltCount; i++)
{
    var y = plateTop + firstBoltOffsetPx + i * boltSpacingPx;
    if (y + boltRadius > plateTop + plateHeightPx)
    {
        break;
    }

    g.DrawEllipse(Pens.Black, boltCenterX - boltRadius, y - boltRadius,
boltRadius * 2, boltRadius * 2);
}

```

```
DrawDimension(g,  
    new PointF(plateLeft + plateThicknessPx + 20, plateTop),  
    new PointF(plateLeft + plateThicknessPx + 20, plateTop + plateHeightPx),  
    $"{plateHeightMm:0.#} mm",  
    false);
```

```
DrawDimension(g,  
    new PointF(beamLeft, beamTop - 25),  
    new PointF(beamLeft + beamRect.Width, beamTop - 25),  
    $"{beamLengthMm:0.#} mm",  
    true);
```

```
using (var borderPen = new Pen(Color.FromArgb(229, 231, 235), 1f))  
{  
    g.DrawRectangle(borderPen, 0, 0, width - 1, height - 1);  
}  
}
```

```
private void DrawFrontView(object sender, PaintEventArgs e)  
{  
    if (frontViewPanel == null) return;  
  
    var g = e.Graphics;  
    g.SmoothingMode = SmoothingMode.AntiAlias;  
  
    var width = frontViewPanel.ClientSize.Width;  
    var height = frontViewPanel.ClientSize.Height;  
    if (width <= 0 || height <= 0)
```



```
{  
    return;  
}
```

```
const double margin = 80;  
  
var p = geometryParams;  
  
var plateWidthMm = Math.Max(p.PlateWidth * 1000.0, 150);  
  
var plateHeightMm = Math.Max(p.PlateHeight * 1000.0, 200);  
  
var scale = (float)Math.Min((width - margin) / (plateWidthMm + 100), (height -  
margin) / (plateHeightMm + 140));  
  
if (float.IsNaN(scale) || scale <= 0) scale = 0.35f;  
  
  
var plateWidthPx = (float)(plateWidthMm * scale);  
  
var plateHeightPx = (float)(plateHeightMm * scale);  
  
var plateLeft = (float)((width - plateWidthPx) / 2);  
  
var plateTop = (float)((height - plateHeightPx) / 2);  
  
  
var plateRect = new RectangleF(plateLeft, plateTop, plateWidthPx,  
plateHeightPx);  
  
g.FillRectangle(Brushes.WhiteSmoke, plateRect);  
  
g.DrawRectangle(Pens.Gray, plateRect.X, plateRect.Y, plateRect.Width,  
plateRect.Height);  
  
  
var boltRadiusPx = Math.Max((float)(p.BoltDiameter * 1000.0 * scale * 0.4),  
5f);  
  
var boltSpacingPx = Math.Max((float)(p.BoltSpacing * 1000.0 * scale), 30f);  
  
var boltEdgeTopPx = Math.Max((float)(p.BoltEdgeTop * 1000.0 * scale), 25f);  
  
var gaugePx = Math.Max((float)(p.BoltGauge * 1000.0 * scale / 2.0),  
boltRadiusPx * 2);
```

```

for (var i = 0; i < p.BoltCount; i++)
{
    var y = plateTop + boltEdgeTopPx + i * boltSpacingPx;
    if (y > plateTop + plateHeightPx - boltEdgeTopPx)
    {
        break;
    }

    foreach (var offset in new[] { -gaugePx, gaugePx })
    {
        g.DrawEllipse(Pens.DimGray, plateLeft + plateWidthPx / 2 + offset -
            boltRadiusPx, y - boltRadiusPx, boltRadiusPx * 2, boltRadiusPx * 2);
    }
}

```

```

DrawDimension(g,
    new PointF(plateLeft, plateTop - 30),
    new PointF(plateLeft + plateWidthPx, plateTop - 30),
    $"{plateWidthMm:0.#} mm",
    true);

```

```

DrawDimension(g,
    new PointF(plateLeft + plateWidthPx + 20, plateTop),
    new PointF(plateLeft + plateWidthPx + 20, plateTop + plateHeightPx),
    $"{plateHeightMm:0.#} mm",
    false);

```

```

if (p.BoltCount > 1)

```

```

{
    var boltSpanPx = (p.BoltCount - 1) * boltSpacingPx;
    DrawDimension(g,
        new PointF(plateLeft - 40, plateTop + boltEdgeTopPx),
        new PointF(plateLeft - 40, plateTop + boltEdgeTopPx + boltSpanPx),
        $"{{{(p.BoltCount - 1) * p.BoltSpacing * 1000.0):0.#{}} mm",
        false);
}

using (var borderPen = new Pen(Color.FromArgb(229, 231, 235), 1f))
{
    g.DrawRectangle(borderPen, 0, 0, width - 1, height - 1);
}
}

private void DrawDimension(Graphics g, PointF start, PointF end, string label,
bool horizontal)
{
    var stroke = Pens.Gray;
    g.DrawLine(stroke, start, end);

    if (horizontal)
    {
        g.DrawLine(stroke, start.X, start.Y - 5, start.X, start.Y + 5);
        g.DrawLine(stroke, end.X, end.Y - 5, end.X, end.Y + 5);
    }
    else
    {

```

```

        g.DrawLine(stroke, start.X - 5, start.Y, start.X + 5, start.Y);
        g.DrawLine(stroke, end.X - 5, end.Y, end.X + 5, end.Y);
    }

    using (var font = new Font(Font.FontFamily, 10f, FontStyle.Bold))
    {
        var textSize = g.MeasureString(label, font);
        if (horizontal)
        {
            g.DrawString(label, font, Brushes.Gray, (start.X + end.X) / 2 -
textSize.Width / 2, start.Y - textSize.Height - 4);
        }
        else
        {
            g.DrawString(label, font, Brushes.Gray, end.X + 6, (start.Y + end.Y) / 2 -
textSize.Height / 2);
        }
    }
}

private void ShowStatus(string message, bool isError)
{
    if (statusLabel == null) return;
    statusLabel.Text = message;
    statusLabel.ForeColor = isError ? Color.Firebrick : SystemColors.ControlText;
}
}

```

```
internal sealed class DoubleBufferedPanel : Panel
```

```
{  
    public DoubleBufferedPanel()  
    {  
        DoubleBuffered = true;  
        ResizeRedraw = true;  
    }  
}
```

```
internal class GeometryParameters
```

```
{  
    public double BeamLength { get; set; } = 2.6;  
    public double BeamHeight { get; set; } = 0.3;  
    public double BeamThickness { get; set; } = 0.15;  
    public double BeamFlangeThickness { get; set; } = 0.012;  
    public double BeamWebThickness { get; set; } = 0.008;  
    public double ColumnHeight { get; set; } = 0.26;  
    public double ColumnWidth { get; set; } = 0.2;  
    public double ColumnDepth { get; set; } = 0.31;  
    public double ColumnFlangeThickness { get; set; } = 0.02;  
    public double ColumnWebThickness { get; set; } = 0.01;  
    public double ColumnWebWidth { get; set; } = 0.2;  
    public double PlateHeight { get; set; } = 0.3;  
    public double PlateWidth { get; set; } = 0.2;  
    public double PlateThickness { get; set; } = 0.016;  
    public double PlateEdgeHorizontal { get; set; } = 0.04;  
    public double PlateEdgeTop { get; set; } = 0.03;  
    public double PlateEdgeBottom { get; set; } = 0.04;
```

```
public int BoltCount { get; set; } = 4;

public double BoltSpacing { get; set; } = 0.075;

public double BoltDiameter { get; set; } = 0.022;

public double BoltEdgeTop { get; set; } = 0.05;

public double BoltGauge { get; set; } = 0.12;

public double WeldSize { get; set; } = 0.005;

}
```

```
internal sealed class ProfileDefinition
```

```
{

    public string Name { get; }

    public double DepthMm { get; }

    public double WebThicknessMm { get; }

    public double FlangeThicknessMm { get; }

    public double FlangeWidthMm { get; }
```

```
    public ProfileDefinition(string name, double depthMm, double
webThicknessMm, double flangeThicknessMm, double flangeWidthMm)
```

```
{

    Name = name;

    DepthMm = depthMm;

    WebThicknessMm = webThicknessMm;

    FlangeThicknessMm = flangeThicknessMm;

    FlangeWidthMm = flangeWidthMm;

}
```

```
public override string ToString()
```

```
{
```

```
        return Name;
    }
}
```

```
internal static class ProfileLibraryData
{
    public static readonly IReadOnlyList<ProfileDefinition> Profiles = new
List<ProfileDefinition>
    {
        new ProfileDefinition("W 150x13,0", 150, 4.0, 5.0, 100),
        new ProfileDefinition("W 150x18,0", 153, 5.8, 7.1, 102),
        new ProfileDefinition("W 150x22,5", 152, 5.8, 6.6, 152),
        new ProfileDefinition("W 150x24,0", 160, 6.6, 10.3, 103),
        new ProfileDefinition("W 150x29,8", 157, 6.6, 9.3, 154),
        new ProfileDefinition("W 150x37,1", 162, 8.1, 11.6, 156),
        new ProfileDefinition("W 200x15,0", 200, 4.3, 5.2, 100),
        new ProfileDefinition("W 200x19,3", 203, 5.8, 6.5, 102),
        new ProfileDefinition("W 200x22,5", 207, 6.2, 8.5, 133),
        new ProfileDefinition("W 200x26,6", 201, 5.8, 8.4, 133),
        new ProfileDefinition("W 200x31,3", 204, 6.4, 10.2, 134),
        new ProfileDefinition("W 200x35,9", 203, 6.2, 9.5, 165),
        new ProfileDefinition("W 200x41,7", 207, 7.1, 11.5, 166),
        new ProfileDefinition("W 200x46,1", 202, 7.2, 11.0, 203),
        new ProfileDefinition("W 200x52,0", 206, 8.0, 12.6, 204),
        new ProfileDefinition("W 250x17,9", 250, 4.8, 5.3, 101),
        new ProfileDefinition("W 250x22,3", 254, 5.6, 6.8, 102),
        new ProfileDefinition("W 250x25,3", 257, 6.1, 8.3, 146),
        new ProfileDefinition("W 250x28,4", 260, 6.1, 9.8, 147),
```

new ProfileDefinition("W 250x32,7", 251, 6.1, 9.7, 177),
new ProfileDefinition("W 250x38,5", 255, 6.9, 11.9, 178),
new ProfileDefinition("W 250x44,8", 259, 7.9, 14.0, 180),
new ProfileDefinition("W 250x49,1", 249, 7.4, 11.7, 203),
new ProfileDefinition("W 250x54,9", 252, 8.0, 13.5, 204),
new ProfileDefinition("W 250x67,3", 258, 9.1, 16.5, 206),
new ProfileDefinition("W 250x72,9", 252, 8.6, 13.2, 254),
new ProfileDefinition("W 250x80,3", 256, 9.5, 15.2, 255),
new ProfileDefinition("W 250x88,6", 260, 10.4, 17.3, 257),
new ProfileDefinition("W 310x21,0", 303, 5.1, 5.7, 101),
new ProfileDefinition("W 310x23,8", 307, 5.5, 6.7, 102),
new ProfileDefinition("W 310x28,3", 312, 6.6, 8.3, 165),
new ProfileDefinition("W 310x32,7", 314, 7.1, 10.0, 166),
new ProfileDefinition("W 310x38,7", 307, 6.1, 9.7, 165),
new ProfileDefinition("W 310x44,5", 311, 6.9, 11.8, 167),
new ProfileDefinition("W 310x52,0", 315, 7.9, 13.9, 169),
new ProfileDefinition("W 310x52,0 (alt)", 310, 7.5, 13.1, 204),
new ProfileDefinition("W 310x60,0", 309, 7.6, 13.2, 203),
new ProfileDefinition("W 310x74,4", 316, 9.4, 16.5, 206),
new ProfileDefinition("W 310x86,3", 308, 9.4, 15.2, 254),
new ProfileDefinition("W 310x101,0", 314, 10.7, 18.7, 257),
new ProfileDefinition("W 310x118,0", 320, 12.4, 21.3, 260),
new ProfileDefinition("W 310x129,0", 324, 13.6, 23.4, 262),
new ProfileDefinition("W 360x32,9", 352, 6.4, 8.5, 127),
new ProfileDefinition("W 360x39,0", 356, 6.9, 10.3, 128),
new ProfileDefinition("W 360x44,6", 360, 7.6, 12.2, 129),
new ProfileDefinition("W 360x51,0", 353, 7.1, 11.4, 171),
new ProfileDefinition("W 360x51,0 (alt)", 355, 7.9, 11.0, 172),

new ProfileDefinition("W 360x58,0", 357, 8.0, 13.5, 172),
new ProfileDefinition("W 360x64,0", 360, 8.8, 15.4, 173),
new ProfileDefinition("W 360x72,0", 368, 9.8, 17.4, 175),
new ProfileDefinition("W 360x79,0", 355, 8.9, 14.4, 253),
new ProfileDefinition("W 360x79,0 (alt)", 362, 10.0, 16.4, 255),
new ProfileDefinition("W 360x91,0", 360, 10.0, 16.8, 254),
new ProfileDefinition("W 360x101,0", 363, 11.2, 18.3, 256),
new ProfileDefinition("W 360x110,0", 367, 12.2, 20.3, 258),
new ProfileDefinition("W 360x122,0", 359, 11.2, 18.0, 305),
new ProfileDefinition("W 360x134,0", 363, 12.2, 20.1, 306),
new ProfileDefinition("W 360x147,0", 367, 13.3, 22.1, 308),
new ProfileDefinition("W 360x179,0", 375, 16.1, 26.2, 312),
new ProfileDefinition("W 410x38,8", 400, 6.4, 8.5, 140),
new ProfileDefinition("W 410x46,1", 404, 6.9, 10.3, 141),
new ProfileDefinition("W 410x53,0", 408, 7.6, 12.2, 142),
new ProfileDefinition("W 410x60,0", 403, 7.7, 12.8, 177),
new ProfileDefinition("W 410x60,0 (alt)", 408, 8.8, 12.8, 178),
new ProfileDefinition("W 410x67,0", 407, 8.6, 14.7, 178),
new ProfileDefinition("W 410x75,0", 411, 9.7, 16.5, 180),
new ProfileDefinition("W 410x85,0", 417, 10.9, 18.8, 181),
new ProfileDefinition("W 410x85,0 (alt)", 417, 10.9, 17.7, 256),
new ProfileDefinition("W 460x52,0", 449, 7.1, 10.8, 152),
new ProfileDefinition("W 460x60,0", 454, 7.9, 12.7, 154),
new ProfileDefinition("W 460x68,0", 459, 8.9, 14.7, 155),
new ProfileDefinition("W 460x74,0", 457, 8.0, 14.5, 190),
new ProfileDefinition("W 460x82,0", 461, 8.9, 16.5, 191),
new ProfileDefinition("W 460x89,0", 464, 9.5, 18.0, 192),
new ProfileDefinition("W 460x97,0", 467, 10.3, 19.8, 194),

```
new ProfileDefinition("W 460x106,0", 470, 11.2, 21.5, 195),
new ProfileDefinition("W 530x66,0", 525, 8.6, 12.7, 165),
new ProfileDefinition("W 530x72,0", 528, 9.1, 14.0, 166),
new ProfileDefinition("W 530x74,0", 526, 8.5, 13.3, 208),
new ProfileDefinition("W 530x82,0", 530, 9.5, 15.2, 209),
new ProfileDefinition("W 530x85,0", 530, 9.8, 15.6, 209),
new ProfileDefinition("W 530x92,0", 533, 10.2, 17.3, 210),
new ProfileDefinition("W 530x101,0", 536, 11.2, 18.8, 211),
new ProfileDefinition("W 530x109,0", 539, 11.9, 20.3, 212),
new ProfileDefinition("W 530x123,0", 544, 13.5, 22.9, 214),
new ProfileDefinition("W 530x138,0", 550, 14.7, 25.9, 216),
new ProfileDefinition("W 610x82,0", 599, 9.4, 12.7, 178),
new ProfileDefinition("W 610x92,0", 603, 10.3, 14.7, 179),
new ProfileDefinition("W 610x101,0", 605, 10.9, 16.1, 228),
new ProfileDefinition("W 610x113,0", 608, 11.9, 18.0, 229),
new ProfileDefinition("W 610x125,0", 612, 12.8, 20.1, 230),
new ProfileDefinition("W 610x140,0", 616, 14.1, 22.2, 231),
new ProfileDefinition("W 610x153,0", 619, 15.1, 23.9, 232),
new ProfileDefinition("W 610x155,0", 608, 11.9, 19.3, 305),
new ProfileDefinition("W 610x174,0", 613, 13.5, 21.6, 306),
new ProfileDefinition("W 610x195,0", 619, 15.1, 24.5, 308),
new ProfileDefinition("W 610x217,0", 624, 16.5, 27.0, 310)
};
}
}
```